Government Savings

Code Coverage Gate — Executive Decision Brief

Prepared by: Swapnil Patil
QA Automation & Quality Program Lead, Government Savings

Assessment Date: July 21, 2026
Version: 1.0
Classification: Confidential — Internal Use

Decision Summary

As of: 2026-07-21

Audience: Government Savings leadership

Evidence: Repository scan + `verified-metrics-register.csv` + CI YAML review

| Layer | What exists | What is missing |

|-------|-------------|-----------------|

| **Merge hygiene (GitLab)** | Protected `main`, MR pipeline must pass, Snyk, dual approval (incl. senior Code Review), discussion/change-request blocks, dependency/conflict rules, draft/rebase enforcement, conditional auto-merge | Live GitLab API verification **blocked** (expired token) |

| **Test-result gates** | UniteMSC service unit/integration tests in CI; V3 + metadataweb scheduled regression hard-fail | Full GS API/UI regression on every MR |

| **Source-code coverage (A)** | JaCoCo in UniteMSC POMs; reports in CI `report` stage (template) | SonarQube disabled; no central dashboard; no branch-vs-main delta |

| **Coverage merge gate** | Static JaCoCo `check` on some services (0%–90% minimum) | **No** verified coverage-delta MR gate, required status check, or auditable bypass |

Michael Blake Q4–Q6 direct answers: We **cannot** today automatically reject an MR solely because code coverage decreased versus `main`. A controlled bypass group exists for **general merge approvals**, not for a coverage-specific gate with recorded exception reasons. **Zero repositories** have a verified full coverage-delta control chain.

| Item | Finding | Status |

|------|---------|--------|

| UniteMSC CI | `RUN_SONARQUBE: false` on 12 reviewed `.gitlab-ci.yml` files | Verified from repository code |

| JaCoCo thresholds | `unite-mobile2` 90%; `unite-profile` 90%; `unite-account` 50%; most others 0% | Verified from repository code |

| Coverage delta | No `coverage:` regex, `coverage_report`, Codecov, or branch-baseline script in reviewed YAML | Verified absent |

| QA pipelines | `api-test-automation`, `prime-test-automation` — JUnit only, no code coverage | Verified |

| GitLab API | `glab api user` → HTTP 401 expired token | Blocked by access |

Line/branch coverage must stay above a fixed % (e.g., 80%/70%).

| Pros | Cons |

|------|------|

| Simple Maven `jacoco:check` | Punishes legacy repos; not tied to changed code |

| Already partially in POMs | Thresholds inconsistent (0%–90%) |

PR coverage cannot be lower than `main` (overall + changed-code guardrail).

| Pros | Cons |

|------|------|

| Fair to legacy codebases | Requires baseline artifact from `main` build |

| Detects real regressions | Needs pipeline + protected-branch wiring |

New/modified lines must meet threshold; overall may be informational.

| Pros | Cons |

|------|------|

| Focuses on new risk | Tooling slightly more complex |

| Leadership-friendly | Requires diff scope agreement |

Combine Options 2 and 3 in phased rollout:

1. Run unit/integration tests on every MR.

2. Generate JaCoCo XML.

3. Compare branch coverage with `main` baseline (no material regression).

4. Enforce changed-code coverage threshold (repository-specific).

5. Publish visible MR coverage check → make required before merge.

6. Retain existing reviewer and approval rules.

7. Allow only a **named senior exception group** to bypass with documented justification, owner, risk acceptance, and expiration; report monthly.

Do not apply one identical threshold to every repository.

| Attribute | Value |

|---|---|

| **Repository** | `unite-mobile2` (GitLab, JaCoCo 90% POM baseline, high MSC visibility) |

| **Alt** | `unite-profile` or GitHub app repo if leadership prefers GHA |

| **Phase 1 (weeks 1–4)** | Informational MR comment + artifact; no merge block |

| **Phase 2 (weeks 5–8)** | Soft fail; exception path tested |

| **Phase 3 (week 9+)** | Required status check on protected `main` |

| **QA Automation** | Pilot comparison script, reporting, register updates |

| **DevOps/Engineering** | Pipeline template, baseline storage, protected-branch required check, bypass group |

| Risk | Mitigation |

|---|---|

| False positives block delivery | Informational phase; repo-specific thresholds |

| Legacy low coverage | Changed-code focus; gradual threshold increase |

| Metric confusion (A vs B) | Separate registers; leadership training |

| Token/API gaps | Provision read-only GitLab PAT early |

| Workstream | Effort |

|---|---|

| Pilot pipeline + script | 2–3 sprints (DevOps + QA) |

| Register + collectors | 1–2 sprints (QA Automation) |

| Org rollout (14 services) | 2–3 quarters phased |

| Governance + exception process | 2–4 weeks (leadership + QA) |

| Area | Owner |

|------|-------|

| JaCoCo delta logic, register, reports | QA Automation |

| Pipeline templates, required checks, bypass group config | DevOps / Engineering |

| Thresholds, exception policy, denominators | Product / BA / QA Governance |

| qTest/Jira hygiene | QA Governance + BAs |

| Field | Required |

|-------|----------|

| Waiver ID | Yes |

| Approver | Named senior exception group (subset of Code Review) |

| Reason | Business or technical |

| Risk acceptance | Named owner |

| Expiration | Max 30 days (recommended) |

| Audit | Monthly exception register published to leadership |

---

1. **Approve phased coverage-delta pilot** on `unite-mobile2` (informational → required).

2. **Confirm exception-approval group** and audit cadence.

3. **Approve domain-specific metrics** — reject single GS-wide %.

4. **Sponsor API credentials** (GitLab, qTest, Jira) for automated register.

5. **Sponsor QA-1405** Mobile 2 API nightly (business regression gate, separate from code coverage).

Matrix: `03-analysis/repository-code-coverage-gate-matrix.csv` | Plan: `03-analysis/code-coverage-gate-implementation-plan.md`